МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Построение и анализ алгоритмов»

Тема: Задача о коммивояжёре

Студент гр. 4343 _____ Семенчик В.В.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Выданное задание.

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные.

- n — количество городов ($5 \leq n \leq 15$).
- Матрица расстояний между городами размером $n \times n$, где graph[i][j] обозначает расстояние от города i до города j. Если graph[i][j]=0 (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные.

- Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.
- Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.
- Если такого пути не существует, вывести "no path".

Цель работы.

Решить задачу о коммивояжёре, закрепить понимание динамического программирования

Динамическое программирование. Описание алгоритма.

Создаётся матрица dp, где dp[mask][i] – минимальная стоимость посещения всех городов из mask так, чтобы путь заканчивался в городе i. Изначальное вся матрица инициализирована INT_MAX, кроме dp[1 << 0][0] = 0.

Матрица parent, где parent[mask][i] – предпоследний город в самом дешёвом пути, посетившем города из mask и закончившемся в i.

Первый цикл – перебор всех масок (2^n итераций). Для каждой из масок проверяется, что в ней посещён нулевой город (ведь с него начинается путь).

Второй вложенный цикл – перебор всех n городов, кандидатов, чтобы быть предпоследним. Для дальнейшей работы цикла город должен быть а) посещён в этой маске б) существует такой путь.

Третий вложенный цикл – перебор всех n городов. Город должен быть не посещён в текущей маске и должен существовать путь из предыдущего города. Если вычисленная стоимость нового пути (посещение всех городов из маски, заканчивая в городе u + стоимость перехода из u в v) меньше dp[mask|(1<<v)][v] – стоимость записывается в dp.

После работы цикла добавляется переход из последнего города обратно в начало пути – нулевой город.

Исследование сложности.

Таким образом, сложность алгоритма – произведение количества итераций трёх циклов. Временная сложность – O$(2^n * n * n)$ или же $O(2^n * n^2)$.

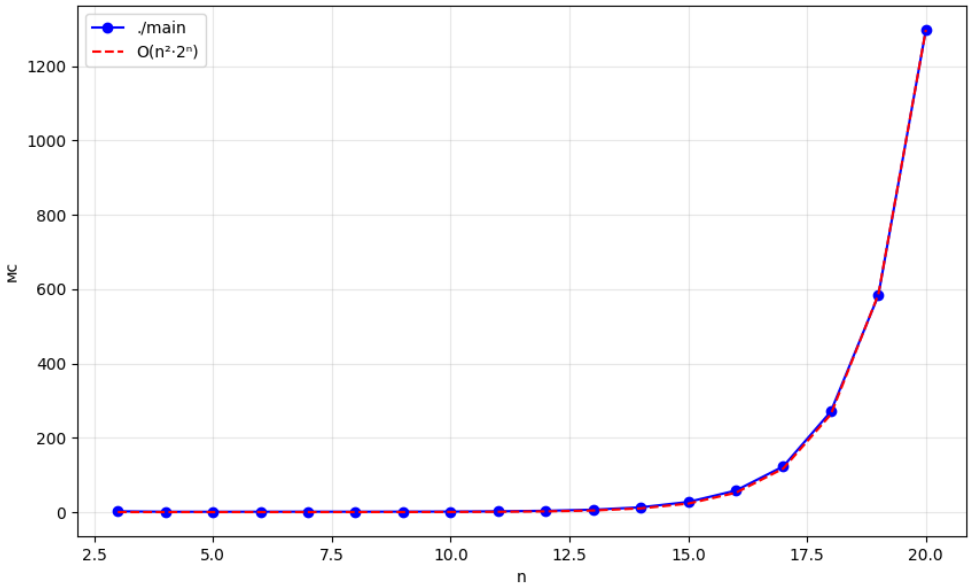

Рисунок 1. Исследование сложности алгоритма

Пространственная сложность – $O(2^n * n)$, что соответствует размеру parent и dp (в нотации не учитывается константа 2).

АЛШ-2. Описание алгоритма.

Алгоритм ALSH-2 представляет собой жадную эвристику для приближённого решения задачи коммивояжёра. Он строит путь, начиная с города 0, и на каждом шаге выбирает следующий город на основе минимальной оценки стоимости value = graph[current][v] + L, где L — вес МОД для ещё не посещённых городов, включая v.

Инициализация: начальный город 0, массив visited отмечает посещённые города, вектор path хранит порядок обхода.

В основном цикле (от 1 до n-1) для текущего города current перебираются все ещё не посещённые города v, в которые есть путь. Для каждого кандидата v временно помечаем его как посещённый, вычисляем L с помощью функции primMST, которая находит вес минимального остовного дерева для множества ещё не посещённых городов (игнорируя visited) классическим алгоритмом Прима с массивом minEdge и флагом inMST, после чего вычисляем value = graph[current][v] + L и выбираем v с минимальным value как следующий город. Выбранный город добавляется в path, помечается как посещённый, обновляется totalCost и current. После посещения всех городов добавляется ребро из последнего города в город 0; если ребра нет — путь невозможен.

Исследование сложности.

Внешний цикл выполняется n-1 раз, на каждом шаге перебирается до n кандидатов, и для каждого кандидата запускается алгоритм Прима сложностью $O(n^2)$, итого $O(n^4)$.

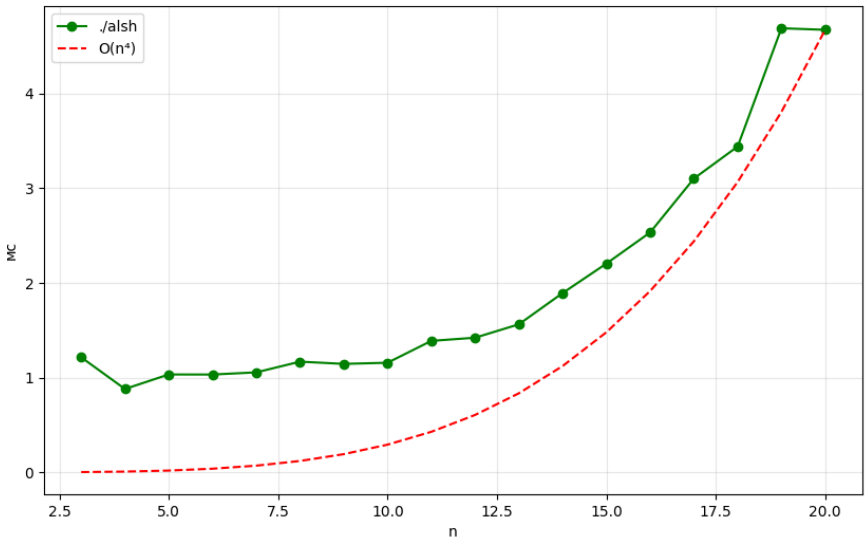

Рисунок 2. Исследование сложности алгоритма

Пространственная сложность - $O(n^2)$

Выводы.

В ходе лабораторной работы была решена задача о коммивояжёре точным и приближённым методом.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#define LOG false

#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>

using namespace std;

int main() {
    int n;
    cin >> n;

    vector<vector<int>> graph(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> graph[i][j];
        }
    }

    vector<vector<int>> dp(1 << n, vector<int>(n, INT_MAX));

    vector<vector<int>> parent(1 << n, vector<int>(n, -1));

    dp[1 << 0][0] = 0;

    for (int mask = 1; mask < (1 << n); mask++) {
        if (!(mask & 1)) continue;

        if (LOG) {
            cout << "Обработка маски: " << mask << " (";
            for (int i = 0; i < n; i++) {
```

```
                    if (mask & (1 << i)) cout << i << " ";
                }
                cout << ")" << endl;
            }

            for (int u = 0; u < n; u++) {
                if (!(mask & (1 << u))) continue;
                if (dp[mask][u] == INT_MAX) continue;

                if (LOG) cout << "Текущий город u=" << u << ", стоимость
пути: " << dp[mask][u] << endl;

                for (int v = 0; v < n; v++) {
                    if (mask & (1 << v)) continue;
                    if (graph[u][v] == 0) continue;

                    int newMask = mask | (1 << v);
                    int newCost = dp[mask][u] + graph[u][v];

                    if (LOG) cout << "Рассматриваем переход из u=" << u
<< " в v=" << v
                                << ", вес=" << graph[u][v]
                                << ", новая маска=" << newMask
                                << ", новая стоимость=" << newCost <<
endl;

                    if (newCost < dp[newMask][v]) {
                        dp[newMask][v] = newCost;
                        parent[newMask][v] = u;

                        if (LOG) cout << "Обновляем: dp[" << newMask <<
"][" << v
                                    << "]. было " << dp[newMask][v]
                                    << ", стало " << newCost << endl;
                        dp[newMask][v] = newCost;
                        parent[newMask][v] = u;
```

```
                    } else if (LOG) {
                        cout << "Не обновляем: " << newCost << " >= " <<
dp[newMask][v] << endl;
                    }
                }
            }
        }

        int fullMask = (1 << n) - 1;
        int minCost = INT_MAX;
        int lastCity = -1;

        for (int i = 1; i < n; i++) {
            if (dp[fullMask][i] == INT_MAX) continue;
            if (graph[i][0] == 0) continue;

            int totalCost = dp[fullMask][i] + graph[i][0];
            if (totalCost < minCost) {
                minCost = totalCost;
                lastCity = i;
            }
        }

        if (minCost == INT_MAX) {
            cout << "no path" << endl;
            return 0;
        }

        vector<int> path;
        path.push_back(0);
        int mask = fullMask;
        int current = lastCity;

        while (current != 0) {
            path.push_back(current);
```

```
            int prev = parent[mask][current];
            mask &= ~(1 << current);
            current = prev;
        }
        path.push_back(0);

        reverse(path.begin(), path.end());

        cout << minCost << endl;
        for (int i = 0; i < path.size(); i++) {
            cout << path[i];
            if (i < path.size() - 1) cout << " ";
        }
        cout << endl;

        return 0;
}
```

Название файла: alsh-2.cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
#include <numeric>

using namespace std;

const int INF = INT_MAX / 2;

int primMST(const vector<vector<int>>& graph, const vector<bool>& visited)
{
    int n = graph.size();
    vector<bool> inMST(n, false);
    vector<int> minEdge(n, INF);

    int totalVertices = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) totalVertices++;
    }

    if (totalVertices <= 1) return 0;

    int start = -1;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            start = i;
            break;
```

```
        }
    }

    if (start == -1) return 0;

    minEdge[start] = 0;
    int mstWeight = 0;

    for (int count = 0; count < totalVertices; count++) {
        int u = -1;
        for (int i = 0; i < n; i++) {
            if (!visited[i] && !inMST[i] && (u == -1 || minEdge[i] <
minEdge[u])) {
                u = i;
            }
        }

        if (u == -1) break;

        inMST[u] = true;
        mstWeight += minEdge[u];

        for (int v = 0; v < n; v++) {
            if (!visited[v] && !inMST[v] && graph[u][v] != 0 && graph[u][v]
< minEdge[v]) {
                minEdge[v] = graph[u][v];
            }
        }
    }

    return mstWeight;
}

pair<int, vector<int>> ALSH2(const vector<vector<int>>& graph) {
    int n = graph.size();

    vector<bool> visited(n, false);
    vector<int> path;

    int current = 0;
    path.push_back(current);
    visited[current] = true;

    int totalCost = 0;

    for (int step = 1; step < n; step++) {
        int bestNext = -1;
        int bestValue = INF;

        for (int v = 0; v < n; v++) {
            if (visited[v]) continue;
            if (graph[current][v] == 0) continue;


            vector<bool> tempVisited = visited;
            tempVisited[v] = true;
```

```
            int L = primMST(graph, tempVisited);

            int value = graph[current][v] + L;

            if (value < bestValue) {
                bestValue = value;
                bestNext = v;
            }
        }

        if (bestNext == -1) return {INF, {}};

        totalCost += graph[current][bestNext];
        current = bestNext;
        path.push_back(current);
        visited[current] = true;
    }

    if (graph[current][0] == 0) {
        return {INF, {}};
    }

    totalCost += graph[current][0];
    path.push_back(0);

    return {totalCost, path};
}

int main() {
    int n;
    cin >> n;

    vector<vector<int>> graph(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> graph[i][j];
        }
    }

    auto result = ALSH2(graph);

    if (result.first >= INF) {
        cout << "no path" << endl;
    } else {
        cout << result.first << endl;
        for (size_t i = 0; i < result.second.size(); i++) {
            cout << result.second[i];
            if (i < result.second.size() - 1) cout << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Название файла: graphics.py

```
import subprocess
```

```
import time
import matplotlib.pyplot as plt
import random

def run_test(exe, n):
    mat = [[random.randint(1, 100) if i != j else 0 for j in range(n)] for
i in range(n)]
    inp = f"{n}\n" + "\n".join(" ".join(map(str, row)) for row in mat)
    start = time.time()
    subprocess.run(exe, input=inp, text=True, capture_output=True)
    return (time.time() - start) * 1000

sizes = list(range(3, 21))
t_main, t_alsh = [], []

for n in sizes:
    tm = [run_test("./main", n) for _ in range(3)]
    ta = [run_test("./alsh", n) for _ in range(3)]
    t_main.append(sum(tm) / len(tm))
    t_alsh.append(sum(ta) / len(ta))

scale_main = t_main[-1] / ((2 ** sizes[-1]) * (sizes[-1] ** 2))
scale_alsh = t_alsh[-1] / (sizes[-1] ** 4)
theor_main = [scale_main * (2 ** n) * (n ** 2) for n in sizes]
theor_alsh = [scale_alsh * (n ** 4) for n in sizes]

plt.figure(figsize=(10, 6))
plt.plot(sizes, t_main, 'bo-', label='./main')
plt.plot(sizes, theor_main, 'r--', label='O(n²·2ⁿ)')
plt.xlabel('n'); plt.ylabel('мс'); plt.legend(); plt.grid(True, alpha=0.3)
plt.savefig('graph1.png', dpi=100)

plt.figure(figsize=(10, 6))
plt.plot(sizes, t_alsh, 'go-', label='./alsh')
plt.plot(sizes, theor_alsh, 'r--', label='O(n⁴)')
plt.xlabel('n'); plt.ylabel('мс'); plt.legend(); plt.grid(True, alpha=0.3)
plt.savefig('graph2.png', dpi=100)
```

Название файла: generate_matrix.cpp

```
#include <iostream>
#include <fstream>
#include <vector>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {
    srand(time(nullptr));

    int N;
    cin >> N;

    vector<vector<int>> g(N, vector<int>(N, 0));

    for (int i = 0; i < N; i++) {
        for (int j = i + 1; j < N; j++) {
```

```
            int w = rand() % 100 + 1; // 1..100
            g[i][j] = w;
            g[j][i] = w;
        }
    }

    ofstream f("input.txt");
    f << N << endl;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            f << g[i][j];
            if (j < N - 1) f << " ";
        }
        f << endl;
    }

    return 0;
}
```

ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ ПРОГРАММЫ

Входные данные	Точный алгоритм	Приближённый алгоритм
3 0 1 0 1 0 1 0 1 0	no path	no path
5 0 1 13 23 7 12 0 15 18 28 21 29 0 33 28 23 19 34 0 38 5 40 7 39 0	78 0 4 2 3 1 0	84 0 4 2 1 3 0
10 0 27 57 88 73 42 68 5 59 68 27 0 10 17 86 72 69 38 7 23 57 10 0 82 64 1 58 3 99 43 88 17 82 0 88 83 24 43 98 72 73 86 64 88 0 68 24 28 55 48 42 72 1 83 68 0 21 75 52 79 68 69 58 24 24 21 0 42 14 48 5 38 3 43 28 75 42 0 27 37 59 7 99 98 55 52 14 27 0 16 68 23 43 72 48 79 48 37 16 0	205 0 7 4 9 8 6 3 1 2 5 0	207 0 7 4 6 3 1 8 9 2 5 0
15 0 5 13 55 2 13 34 47 45 56 62 24 30 36 44 5 0 66 14 92 70 67 25 10 61 91 19 59 31 18 13 66 0 28 100 61 66 56 25 20 57 89 5 3 33 55 14 28 0 61 64 8 42 51 52 7 16 43 28 83 2 92 100 61 0 20 37 95 10 7 53 92 24 81 43 13 70 61 64 20 0 36 98 99 60 69 55 49 73 10 34 67 66 8 37 36 0 33 85 25 41 78 28 44 37 47 25 56 42 95 98 33 0 43 38 64 25 57 53 20 45 10 25 51 10 99 85 43 0 66 59 72 10 35 4 56 61 20 52 7 60 25 38 66 0 52 70 1 50 82 62 91 57 7 53 69 41 64 59 52 0 21 57 82 94 24 19 89 16 92 55 78 25 72 70 21 0 18 14 30	163 0 5 14 8 1 7 6 3 10 11 13 2 12 9 4 0	233 0 4 9 12 2 13 11 7 6 5 14 8 1 3 10 0

30 59 5 43 24 49 28 57 10 1 57 18 0 42 6 36 31 3 28 81 73 44 53 35 50 82 14 42 0 60 44 18 33 83 43 10 37 20 4 82 94 30 6 60 0		